# AI-Based Phishing Detection SOC Assistant

Academic Project Final Course Report & Engineering Documentation

**Context:** Cybersecurity AI Pipeline Prototype
**Deployment Mode:** Local / Privacy-Safe / Non-API Dependent

---

## 1. Problem Statement / Project Goal

In modern Security Operations Centers (SOCs), email-based phishing remains one of the most pervasive and high-risk vectors for initial organizational compromise. Security analysts are routinely inundated with alerts regarding suspicious emails, leading to severe alert fatigue and delayed triage windows.

The goal of this project is to build a lightweight, locally running **SOC Assistant** designed to automate the initial analysis of suspicious emails. The system evaluates phishing risks, extracts key indicators of compromise (IoCs), maps tactics to the MITRE ATT&CK framework, explains why an email is risky, provides customized analyst recommendations, and logs telemetry safely without cloud exposure.

### Core System Constraints

- **Data Privacy:** Write safe audit logs using cryptographic hashing (SHA256) and text metadata, completely avoiding the storage or leakage of raw email bodies.
- **Zero External Dependencies:** Run entirely locally without requiring cloud-based LLM support or external API keys, mitigating data leakage, operational costs, and vendor lock-in risks.
- **Human-in-the-Loop:** Recommend analyst-review actions exclusively. The tool serves as an advisory companion and explicitly avoids automated destructive actions (such as email deletion or credential resets) without certified human sign-off.

## 2. Dataset and Data Preparation

### Dataset Source

The system utilizes the **CEAS_08 Phishing Email Dataset** sourced from Kaggle. For reproducibility and baseline consistency, the project isolates the `CEAS_08.csv` file from the broader collection. The raw file contains 39,154 rows and 7 structural columns: `sender`, `receiver`, `date`, `subject`, `body`, `label`, and `urls`.

### Local Data Flow Pipeline

The architecture processes data through a deterministic, reproducible local lifecycle, ensuring that data files remain local and omitted from version control via `.gitignore` configuration:

```
data/raw/CEAS_08.csv └──> src/preprocessing.py └──> data/processed/ceas08_clean.csv
└──> src/train_baseline.py └──> models/phishing_baseline.joblib
```

### Preprocessing and Feature Engineering

During the execution of `src/preprocessing.py`, the dataset is cleaned, dropping 69 corrupt or null entries, leaving a final structured output of **39,085 rows** and 9 engineered columns:

| Engineered Column | Data Type | Description / Transformation Logic |
|---|---|---|
| `sender` | Text | Raw sender email metadata address. |
| `subject` | Text | Raw email subject text line. |
| `body` | Text | Raw email body content. |
| `label` | Binary | Target variable (1 = Phishing/Spam, 0 = Benign/Legitimate). |
| `has_url` | Binary | Transformed from the raw binary `urls` column to explicitly flag presence. |
| `text` | Text | Combined and normalized string concatenation of `subject` and `body`. |
| `text_length` | Integer | Total character count of the combined `text` field. |
| `subject_length` | Integer | Total character count of the `subject` field. |
| `body_length` | Integer | Total character count of the `body` field. |

## 3. Exploratory Data Analysis (EDA)

The core findings preserved in `reports/eda_summary.md` shaped both the machine learning architecture and the secondary heuristic rule engines:

- **Data Integrity:** Missing and corrupted values were confined to 69 records, proving that the raw dataset provides an exceptional foundation without broad imputation requirements.
- **Label Balance:** The cleaned dataset exhibits a healthy distribution between benign (0) and phishing (1) messages, supporting highly stratified train/test partitions.
- **Structural Variations:** Phishing emails regularly demonstrated significant, distinct variations in text length, subject brevity, and character counts compared to standard corporate correspondence.
- **Tokenization Insights:** While specific tokens and URL presences strongly correlate with malicious activity, relying purely on text-based statistical frequencies was deemed insufficient due to advanced social

engineering tactics that replicate corporate templates. This discovery justified a hybrid approach that joins an ML baseline with rule-based security validation.

## 4. Model Design

To satisfy the core requirement of a lightweight, explainable, and locally deployable system, a traditional natural language processing pipeline was established.

- **Feature Extraction:** A Term Frequency-Inverse Document Frequency (`TfidfVectorizer`) module converts the normalized combined text into a numerical matrix, capturing token relevance across the dataset while filtering out high-frequency stop words.
- **Classification Algorithm:** A `Logistic Regression` classifier processes the feature vector. This model was selected over deep learning or complex ensemble architectures because it provides direct, mathematically traceable feature weights (coefficients), requires minimal processing overhead, and executes near-instantaneous inference on lightweight commodity hardware.

## 5. Model Training and Evaluation Results

The machine learning baseline was compiled locally via `src/train_baseline.py` using a stratified validation partition.

### Evaluation Metrics

The model demonstrated exceptional standard diagnostic metrics during testing:

| Metric | Value Obtained |
|---|---|
| **Accuracy** | 0.9946 |
| **Precision** | 0.9968 |
| **Recall** | 0.9936 |
| **F1-score** | 0.9952 |

### Confusion Matrix Breakdown

The evaluation stage produced the following matrix:

```
[[3439   14]
 [  28 4336]]
```

To align with strict SOC operational methodologies, the outputs are interpreted as follows:

- **True Negatives (TN) [3,439]:** Legitimate business messages correctly identified as benign, minimizing false disruptions to workflow channels.
- **True Positives (TP) [4,336]:** Active phishing threats correctly caught and passed into downstream contextual analysis blocks.
- **False Positives (FP) [14]:** Benign communications incorrectly flagged as phishing. A low false-positive count prevents alert fatigue inside the SOC team.
- **False Negatives (FN) [28]:** Active phishing messages missed by the machine learning algorithm.

> **Operational Risk Context**
>
> In cybersecurity, False Negatives present the highest technical risk. Missed phishing attempts allow threats to infiltrate the network. Because the baseline ML model missed 28 attacks, the assistant is designed as a hybrid system, ensuring that rule-based indicator checks can catch malicious attributes even if the statistical model fails.

## 6. System Architecture and Workflow

The pipeline operates as a sequential architecture, handling each incoming email as an isolated, state-free transaction. The workflow passes telemetry down the chain to dynamically build out defensive context:

```
Email Input → Preprocessing → Phishing Classifier → Indicator Extraction → MITRE ATT&CK Mapping → Risk Scoring / Judging → Explanation → Recommendation → Audit Log → Streamlit Dashboard
```

The core entry point `src/pipeline.py` exposes the central function `analyze_email`, which abstracts this sequence away from the dashboard and outputs a unified, JSON-compatible python dictionary containing: `prediction`, `indicators`, `mitre_mappings`, `risk`, `explanation`, `recommendations`, `audit_logged`, and `input_metadata`.

## 7. Indicator Extraction and MITRE ATT&CK Mapping

Implemented inside `src/indicator_extractor.py` and `src/mitre_mapper.py`, this stage introduces deep cyber threat intelligence context into the process.

### Indicator Extraction

The extraction engine operates deterministically using pre-compiled regular expressions and keyword maps, running completely locally without API dependencies. It scans the email components for:

- Embedded URLs, external domains, and raw IP addresses.

- Urgent or coercive language patterns (e.g., "immediate action", "account suspended").
- Credential harvesting triggers and financial/payment language.
- Attachment mentions, shortened URL patterns, and suspicious sender alignment signals.

### MITRE ATT&CK Mapping

Evidence identified during indicator extraction is correlated against documented adversary tactics and techniques. The mapper is engineered to be conservative, mapping exclusively when clear evidence supports the alignment:

- **T1566.002 (Spearphishing Link):** Triggered upon detecting explicit external web hyperlinks combined with coercive language context.
- **T1566.001 (Spearphishing Attachment):** Assigned when textual text context explicitly flags file inclusions or download prompts.
- **T1204.001** / **T1204.002 (User Execution):** Evidenced alongside malicious links or files requiring human click interaction.
- **T1056.003 (Input Capture: Web Portal Capture):** Triggered when credential harvesting language requests corporate credential entry.
- **T1102 (Web Service):** Applied selectively when external indicator structures point toward unverified public hosting assets.

## 8. Risk Scoring / Judging Module

The `src/risk_scoring.py` module calculates a dynamic composite risk rating spanning an integer scale of **0 to 100**. This score represents a hybrid compromise between machine learning inference and rule-based heuristics.

The scoring engine aggregates variables including the phishing model's raw probability distribution, total link counts, presence of URL shorteners, raw IP usage, urgent semantics, credential harvesting requests, financial keywords, and the density of discovered MITRE ATT&CK techniques. The numerical value maps to four strict organizational risk tiers:

- **0 – 24 (Low Risk):** Safe message with minimal anomalies; passes baseline validation.
- **25 – 49 (Medium Risk):** Shows minor semantic warnings or standard links; requires passive review.
- **50 – 74 (High Risk):** Demonstrates definitive phishing markers or clear malicious model validation.
- **75 – 100 (Critical Risk):** Combines highly anomalous indicators, strong machine learning confidence, and confirmed ATT&CK techniques.

## 9. Explanation and Recommendations

The system bridges automated analysis and human action using text synthesizers localized in `src/explainer.py` and `src/recommendations.py`.

### Explanation Generation

The explainer constructs an objective textual summary that outlines the machine learning model's specific classification label, structural statistical confidence, observed indicators, matched MITRE methodologies, and the dynamic risk score calculation. This allows analysts to immediately verify the underlying rationale behind high-risk alerts.

### Actionable Recommendations

To adhere to standard safety guidelines, the system operates as a pure decision-support layer. It provides targeted action items for manual review, such as conducting header inspection, validating external domain registration ages, and isolating endpoints via independent EDR platforms. The system explicitly avoids automated, destructive steps like mailbox modification, domain blocks, or account lockout actions without human approval.

## 10. Audit Logging and Safety

Data privacy and compliance protocols are strictly maintained via `src/audit_logger.py`. The application maintains an absolute wall between analysis text and disk persistence.

> **Privacy Protection Architecture**
>
> The audit log, saved locally under `logs/audit_log.csv`, completely excludes raw email subjects, headers, or body text. Instead, it securely registers transactions using a cryptographic SHA256 signature of the email body combined with technical metadata fields.

The parameters written to disk are strictly limited to: timestamp, SHA256 email hash, string length metrics, classification prediction, model probability, evaluation confidence, raw risk score, mapped risk tier, indicator flags, MITRE technique arrays, and recommendation summaries. Operational tests using verification utilities confirm that searching the log file for explicit email text strings returns zero hits.

## 11. Streamlit Dashboard Demo

The user-facing ecosystem is exposed via a local web interface built inside `dashboard/app.py` and executed via `streamlit run dashboard/app.py`.

The interface provides a dual input layout: a manual text field for pasting raw email bodies, and a catalog containing 20 preloaded demonstration templates (10 benign corporate messages and 10 phishing-style engineering examples using fake domains). Upon clicking the analysis action trigger, the interface renders a dynamic security dashboard displaying: model probability distributions, numeric gauge representations of risk metrics, structured summaries of threat indicators, tabular MITRE ATT&CK alignments, clear textual explanations, checklist recommendations, and a direct preview of the privacy-safe local audit log.

## 12. Morpheus and Course Lab Connection

This project follows the pipeline-based cybersecurity AI idea introduced in the NVIDIA Morpheus course. Morpheus is designed for building AI pipelines that ingest security data, preprocess it, run machine learning inference, enrich results with security context, and produce SOC-ready outputs.

Although this prototype does not directly deploy the NVIDIA Morpheus runtime, it applies the same architecture at a lightweight local scale. The suspicious email is treated as a security event. The system preprocesses the email text, applies a phishing detection model, extracts indicators, maps evidence to MITRE ATT&CK, calculates risk, explains the result, recommends analyst actions, and logs the event safely.

The project also reuses concepts from the course labs: Lab2 ideas are used for dataset preparation, preprocessing, model training, and evaluation; Lab3 ideas are used for indicators, risk scoring, JSON-style output, and recommendations; Lab1 ideas are used for CTI-style interpretation and MITRE ATT&CK mapping; and Lab4/Lab5 ideas are used for SOC workflow, dashboard demonstration, pipeline-style outputs, and audit logging.

## 13. Limitations and Conclusions

**Limitations:**
This project is a lightweight prototype and not a production SOC platform. The baseline model was trained and evaluated on the CEAS_08 dataset, so performance on real organizational email traffic may be different. The model may miss phishing emails that use new wording, images, attachments, or social engineering patterns not represented in the dataset.

The indicator extraction and risk scoring modules are deterministic and rule-based. This makes the system explainable and safe, but it can also create false positives when benign emails contain words such as password, account, invoice, or update. The MITRE ATT&CK mapping is intentionally conservative and only maps techniques when evidence is present.

The dashboard is an MVP demonstration. It analyzes manually entered email text and safe sample emails, but it is not integrated with a real mailbox, SIEM, SOAR platform, or NVIDIA Morpheus streaming pipeline. The audit logger stores hashes and metadata only, not full email bodies, which improves privacy but limits forensic reconstruction.

**Conclusions:**
The project demonstrates how AI and cybersecurity concepts can be integrated into a practical SOC assistant. The machine learning model provides phishing probability, while rule-based indicators, MITRE ATT&CK mapping, risk scoring, explanations, and recommendations turn the prediction into analyst-friendly CTI context.

The system successfully supports the required workflow: email input, preprocessing, phishing classification, indicator extraction, MITRE mapping, risk scoring, explanation, recommendation, audit logging, and Streamlit dashboard visualization. It works locally without API keys and avoids automatic destructive actions. Overall, the prototype shows that a simple explainable AI model can be combined with SOC-style reasoning to support phishing triage.